
Module 1: RPA Engine, Research & Design

OmniSight module doc | Prepared for Juhi Thacker | Covers roadmap Week 1, Days 1-4

1. Purpose of this Module

The RPA engine is the foundation everything else sits on. It reads a workflow written in JSON, runs each step in a real browser, takes a screenshot after every step, and logs the result. It does **no healing and no vision** yet. Its job is to run a workflow reliably, expose clean hooks for the other modules, and break in a way that is easy to detect.

Done when: the demo workflow runs end to end with plain selectors, and renaming one ID in the test app makes the bot fail visibly and log why.

2. Research Findings

2.1 Playwright is the right base

- Playwright **locators** find elements at the moment of interaction, wait for them to be ready, and retry on failure. This is different from raw selectors, and it means your engine needs almost no manual sleep logic.
- Actions such as click and fill **auto-wait** for the element to be visible and actionable. Use explicit waits only for things like a network response or URL change.
- The recommended order of locator types is **user-facing first**: `getByRole`, `getByLabel`, `getByText`, `getByPlaceholder`, then `getByTestId`, with CSS and XPath as a last resort. Recent 2026 guides also mention a `locator.normalize()` helper that converts brittle CSS or XPath into cleaner role or test-ID locators. Check that it exists in the version you install before relying on it.
- Playwright can **record actions with codegen**, which you will reuse in Week 4 for record-and-replay instead of writing a recorder from scratch.

2.2 ARIA snapshots help later modules

- Playwright can serialize the **accessibility tree** of a page or element as YAML (roles, accessible names, nesting). It survives class renames and markup changes better than raw DOM.
- Tools built for AI agents already use this idea: they take an ARIA snapshot, give each element a short reference ID, and resolve that ID back to a Playwright locator before acting.
- Reports on Playwright's AI-oriented snapshot mode say it is shaped for model prompts and can include element bounding boxes. This is promising for your perception module, but verify the option names in the Node.js API before designing around it.
- **Impact on Module 1:** the engine should resolve elements through one function, `resolveElement(step)`, so Module 2 and Module 3 can plug in better strategies later without changing the runner.

2.3 Version note

Sources found during research mention Playwright releases in the 1.55 to 1.60 range. Run `npm info playwright version` on the day you start and pin that version in `package.json`.

3. Design Decisions

Decision	Choice	Reason
Language	TypeScript on Node.js	Matches existing Node/Express skills, and robotjs works for desktop later
Workflow format	JSON	Simple to write, store, record into, and generate from the planner
Element targeting	Multiple locator candidates per step	Gives the healer something to fall back on without a rewrite
Waiting	Rely on Playwright auto-wait	No custom sleeps
Logging	SQLite via <code>better-sqlite3</code>	Zero setup, one file, easy to query from the dashboard
Screenshots	PNG file per step, path stored in the database	Keeps the database small

4. Workflow JSON Format

Each step has an action, a target, optional value, and an **expected outcome** that the Verifier module will check later.

```
{
  "name": "buy-shoes",
  "baseUrl": "http://localhost:8000",
  "steps": [
    { "id": 1, "action": "goto", "url": "/" },
    {
      "id": 2,
      "action": "click",
      "description": "the Login button",
      "target": { "css": "#login-btn", "text": "Login", "role": "button" },
      "expect": "Login form is visible"
    },
    {
      "id": 3,
      "action": "type",
      "description": "the email field",
      "target": { "css": "#email", "label": "Email" },
      "value": "test@test.com",
      "sensitive": false
    }
  ]
}
```

Why these fields matter

- `description` is plain English. Module 2 uses it to find the element with no selector.
- `target` holds several locator candidates. Module 4 (healing) tries them in order.
- `expect` is what the Verifier checks after the step.
- `sensitive` marks fields such as passwords so the safety layer can mask them in logs and screenshots.

Supported actions in Week 1: `goto`, `click`, `type`, `select`, `wait`, `extract`.

5. Step Runner Design

The runner is a simple loop with clear hook points:

1. Load and validate the workflow JSON (reject unknown actions).

2. Create a run record in SQLite.
3. For each step:
 - **beforeStep** hook (later used by the safety layer)
 - `resolveElement(step)` finds the element from the target candidates
 - perform the action
 - **afterStep** hook (later used by the Verifier)
 - capture a screenshot and log the step result
4. On failure, record the error type, message, and screenshot, then mark the run failed. In Module 4 this is where the healer plugs in.
5. Close the browser and finalize the run record.

Keep these rules from day one

- The runner never contains healing logic. It only calls hooks.
- Every error is classified, for example `ELEMENT_NOT_FOUND`, `TIMEOUT`, `NAVIGATION_FAILED`, or `ACTION_FAILED`. The healer needs this later.
- Each step has a timeout so one stuck step cannot hang the whole run.

6. SQLite Schema (starter)

Table	Key columns
runs	id, workflow_name, status, started_at, finished_at
steps	id, run_id, step_index, action, description, status, error_type, error_message, duration_ms, screenshot_path
heals	added later in Module 4 (strategy, old locator, new locator, confidence)
locator_memory	added later in Module 5

Creating the `steps` table with an `error_type` column now saves a migration later.

7. Suggested Folder Layout for this Module

```
engine/
  workflow.ts      # types and JSON validation
  runner.ts        # main loop and hooks
  actions.ts       # goto, click, type, select, wait, extract
  resolve.ts       # resolveElement(step), plain selectors for now
  logger.ts        # SQLite and screenshot saving
workflows/
  buy-shoes.json
```

8. Build Checklist

Days 1-2

- Initialize the Node.js/TypeScript project and install Playwright
- Define the workflow types and write `buy-shoes.json`
- Implement actions and the runner loop
- Add SQLite logging and per-step screenshots

Days 3-4

- Run the workflow end to end with plain CSS selectors
- Apply mutation 1 (rename an ID) in the test app and confirm the bot fails with a classified error
- Apply mutations 2 and 3 (changed text, moved button) and note which failures are detected

- Write the first devlog entries

9. Risks and Open Questions

- **Over-building:** it is tempting to add retries or fallbacks in the runner. Resist this. That belongs to Module 4.
- **Flaky first runs:** make sure the test app is running and seeded with the same data each time.
- **Open question:** should each workflow run in a fresh browser context? Recommended: yes, so state from one run never leaks into the next.
- **Open question:** where do screenshots live? Recommended: a `data/screenshots/<run_id>/` folder, ignored by Git.

10. Sources Consulted

- BrowserStack guides on Playwright for JavaScript and Playwright locators (2026)
- TestDino Playwright locators guide
- QASkills.sh articles on what is new in Playwright 2026 and on ARIA snapshots
- Playwright ARIA snapshot format notes and the Rust bindings documentation for AI-mode snapshots
- Forum write-up on how the agent-browser tool uses ARIA snapshots with reference IDs

11. Next Module

Module 2, **Perception Layer** (DOM simplifier, vision element finder, and Verifier), builds directly on the `resolveElement` hook and the `expect` field defined here.